REACTNATIVEPLAYGROUND · LEARNING GUIDE · STEP 2 OF 2

Redux cart: build it locally, then switch to the API

A step-by-step guide to a Redux Toolkit cart for the Route app. Part 1 keeps the cart in local state, using the real products HomeTab already loads. Part 2 moves the same cart to the real cart APIs without changing a single screen.

October 27, 2026 · Redux Toolkit 2.13 · react-redux 9.3 · React Native 0.87 · Every file in this guide was typechecked, bundled and tested

The plan

> **Do step 1 first:** this guide starts from the **Home APIs guide**. HomeTab must already load categories and products from the API, and `ProductDto`, `listResponse.ts` and `errors.ts` must exist. The local cart stores those real `ProductDto`s.

The contract with the backend

Every cart endpoint returns **the same response**, the shape `GET /cart` returns today, where each line's `product` is a full object:

Endpoint	What it does	Response today	Response after the backend change
`GET /cart`	Load the cart	Full shape	Full shape
`POST /cart` `{productId}`	Add 1 of a product	`product` is only an id	Full shape
`PUT /cart/:productId` `{count}`	Set a count	Full shape	Full shape
`DELETE /cart/:productId`	Remove a product	Full shape	Full shape

Until the backend change ships, the app calls `GET /cart` right after every `POST /cart`. That workaround lives in **one function** (section 2.3), so removing it later is a two-line change.

Why build it locally first?

- **Learn Redux without network problems.** No tokens, no 401s, no loading states. Just state, actions and reducers.
- **The local cart already uses the real models.** The state is a real `CartResponse`, and the products are the real `ProductDto`s HomeTab gets from `GET /products`. Only the *cart* is local. Switching to the cart API means swapping where the cart data comes from, not reshaping it.

- **The screens never change.** The API version keeps the same action names (`addToCart`, `updateCount`, `removeFromCart`, `clearCart`) and the same selectors. `HomeTab`'s product cards, the cart badge and the cart screen are written once.

Files you'll create or change

File	Part 1 (local cart)	Part 2 (cart API)
`src/theme/colors.ts`	new	unchanged
`src/api/models/response/cartResponse.ts`	new	unchanged
`src/store/store.ts`, `src/store/hooks.ts`	new	unchanged
`src/store/cart/cartSlice.ts`	new (local reducers)	**replaced** (thunks)
`src/common/QuantityControl.tsx`	new	unchanged
`src/common/AddToCartButton.tsx`	new	updated (spinner + error toast)
`src/screens/cart/CartScreen.tsx`	new	updated (loads on open)
`App.tsx`, `RootNavigation.tsx`	updated	unchanged
`HomeTab.tsx`	updated (add button + badge)	unchanged
`src/api/apiServices.ts`	unchanged	updated (exports client, sends token)
`src/api/cartService.ts`	-	new
`MainTabs.tsx`, `ProfileTab.tsx`	unchanged	updated (load cart, logout)

From step 1 and unchanged here: `productResponse.ts`, `listResponse.ts`, `errors.ts`, and `getProducts()` / `getCategories()` in `apiServices.ts`.

How this guide was checked
I built both parts on top of the step 1 code, in copies of your current project. Every code block marked with a file path is the **exact** file from those copies; the PDF builder inserts the files directly, so nothing was retyped.

- **TypeScript and Metro:** both parts typecheck with no new errors and build a full production Metro bundle.
- **Part 1 reducer test** (in Node, with 3 products fetched live from `GET /products`): add/add/add-again, counts, the number of different products, totals, count → 0 removes, remove, clear. All 8 checks pass.
- **Part 2 end-to-end test** (the real thunks and store against a fake server that behaves like Route): `fetchCart`; add = `POST` then `GET`; spinner on and off; PUT on a count change; count 0 = DELETE; a 401 rejects with the server's message; logout clears. All 11 checks pass.
- **Server pricing rule** checked against your real cart: 11×4829 + 1629 + 199 + 11×149 + 5×149 + 499 = 57,830 = `totalCartPrice`. Each line uses `price`, not `priceAfterDiscount`.

Not done: running on a simulator, and calling the live cart endpoints (they need your logged-in account).

0 · Redux in five minutes

The pieces

Redux	What it is	Flutter (Bloc / Riverpod)
store	One object holding all app state	`MultiBlocProvider` / `ProviderScope`
slice	One feature's state + the functions that change it	a `Cubit` / `Notifier`
action	A plain object describing what happened: `{type: "cart/addToCart", payload: product}`	calling `cubit.addToCart(product)`
reducer	`(state, action) => newState`: the only place state changes	the method body that emits the new state
dispatch	Sends an action to the store	`context.read<CartCubit>()…`
selector	A function that reads one value from the state	`context.select` / `ref.watch(p.select(…))`
thunk (Part 2)	An async action: calls the API, then its result goes to a reducer	an `async` Cubit method
Provider	Makes the store reachable from every component	`BlocProvider` at the root

One tap, end to end

```
Tap +
  → dispatch(addToCart(product))          the button sends an action
  → store runs cartSlice's addToCart       the reducer calculates the new CartResponse
  → state.cart.response changes
  → every useAppSelector re-runs           (button count, cart badge, cart screen)
  → only components whose value changed re-render
```

Flutter analogy: the action is the "event" in Bloc, and the reducer is `on<AddToCart>((event, emit) => …)`. Redux keeps "what happened" (action) separate from "how state changes" (reducer). Redux Toolkit generates the actions from your reducers, so you only write the reducers.

Part 1 · The local cart

1.1 · Install

Both packages are pure JavaScript, so you don't need `pod install` or a native rebuild:

```
yarn add @reduxjs/toolkit react-redux
```

- **@reduxjs/toolkit** (RTK): `configureStore`, `createSlice` and `createAsyncThunk`. It's the official way to write Redux today.
- **react-redux**: connects the store to React with `<Provider>`, `useSelector` and `useDispatch`.

1.2 · Colors

New components can't import `blue` from `HomeTab.tsx`: `HomeTab` imports them, so it would be a circular import and `blue` would be `undefined` when their styles are created. A colors file with no imports can be used by anyone.

> src/theme/colors.ts

```
// App colors live here (no imports), so any file can use them without circular imports.
export const colors = {
    primary: "#004182",
    white: "#FFFFFF",
    border: "#00418233",
    danger: "#E53935",
} as const;
```

1.3 · The cart model

The product model, `ProductDto` in `productResponse.ts`, comes from step 1. The cart model builds on it.

The shape every cart API returns

> src/api/models/response/cartResponse.ts

```
import {ProductDto} from "./productResponse.ts";

// The product inside a cart line: the fields GET /cart sends.
// Pick<T, keys> = a new type with only those keys of T, so field types stay in sync with
ProductDto.
export type CartProductDto = Pick<ProductDto,
    "_id" | "id" | "title" | "quantity" | "imageCover" | "category" | "brand" | "subcategory" |
"ratingsAverage">;

export interface CartItemDto {
    _id: string;              // the cart line's own id (not the product's)
    count: number;
    price: number;            // the product's `price` (not priceAfterDiscount)
    product: CartProductDto;  // always the full object: the shape the backend will use for every
```

```
cart API
}

export interface CartDto {
    _id: string;
    cartOwner: string;
    products: CartItemDto[];
    totalCartPrice: number;
}

// The one response shape for GET, POST, PUT and DELETE /cart
export interface CartResponse {
    status: string;
    message?: string;
    numOfCartItems: number;
    cartId: string;
    data: CartDto;
}

// Builds the cart version of a product, the same fields GET /cart returns
export function toCartProduct(product: ProductDto): CartProductDto {
    return {
        _id: product._id,
        id: product.id,
        title: product.title,
        quantity: product.quantity,
        imageCover: product.imageCover,
        category: product.category,
        brand: product.brand,
        subcategory: product.subcategory,
        ratingsAverage: product.ratingsAverage,
    };
}
```

- **`Pick<ProductDto, …>`** builds the cart's product type from `ProductDto`, keeping only the fields `GET /cart` sends. If a field's type changes in `ProductDto`, the cart type follows.
- **The cart's product has no `price`**; the price lives on the line (`item.price`). Because of the `Pick`, `item.product.price` is a compile error, which stops you from reading it from the wrong place.
- **`product: CartProductDto`** is always an object. That's the backend contract, so no `string | object` checks are needed anywhere.
- **`toCartProduct`** converts a full product into the cart's version, so a locally added line looks exactly like one from the server.

1.4 · The cart slice (local)

> src/store/cart/cartSlice.ts

```
import {createSlice, PayloadAction} from "@reduxjs/toolkit";
import {ProductDto} from "../../api/models/response/productResponse.ts";
```

```
import {CartResponse, toCartProduct} from "../../api/models/response/cartResponse.ts";
import type {RootState} from "../store.ts";

interface CartState {
    response: CartResponse;   // exactly what GET /cart returns
}

// An empty cart in the server's shape
function emptyCart(): CartResponse {
    return {
        status: "success",
        numOfCartItems: 0,
        cartId: "local",
        data: {_id: "local", cartOwner: "local", products: [], totalCartPrice: 0},
    };
}

const initialState: CartState = {
    response: emptyCart(),
};

// Recalculates the totals the way the server does:
// numOfCartItems = number of different products, totalCartPrice = sum of count × price
function recalculate(cart: CartResponse) {
    cart.numOfCartItems = cart.data.products.length;
    cart.data.totalCartPrice = cart.data.products.reduce((sum, item) => sum + item.count *
item.price, 0);
}

const cartSlice = createSlice({
    name: "cart",
    initialState,
    reducers: {
        addToCart(state, action: PayloadAction<ProductDto>) {
            const product = action.payload;
            const lines = state.response.data.products;
            const existing = lines.find(item => item.product._id === product._id);
            if (existing) {
                existing.count += 1;   // the server's POST also adds 1 when it's already there
            } else {
                lines.push({
                    _id: `local-${product._id}`,   // the server creates the real line id
                    count: 1,
                    price: product.price,
                    product: toCartProduct(product),
                });
            }
            recalculate(state.response);
        },
```

```
        updateCount(state, action: PayloadAction<{ productId: string; count: number }>) {
            const {productId, count} = action.payload;
            const data = state.response.data;
            if (count <= 0) {
                data.products = data.products.filter(item => item.product._id !== productId);
            } else {
                const item = data.products.find(line => line.product._id === productId);
                if (item) item.count = count;
            }
            recalculate(state.response);
        },
        removeFromCart(state, action: PayloadAction<string>) {
            const data = state.response.data;
            data.products = data.products.filter(item => item.product._id !== action.payload);
            recalculate(state.response);
        },
        clearCart() {
            return initialState;   // e.g. on logout
        },
    },
});

export const {addToCart, updateCount, removeFromCart, clearCart} = cartSlice.actions;
export default cartSlice.reducer;

// ---- selectors: screens only read the cart through these ----
export const selectCartItems = (state: RootState) => state.cart.response.data.products;
export const selectCartCount = (state: RootState) => state.cart.response.numOfCartItems;
export const selectCartTotal = (state: RootState) => state.cart.response.data.totalCartPrice;
export const selectCount = (state: RootState, productId: string) =>
    state.cart.response.data.products.find(item => item.product._id === productId)?.count ?? 0;
```

How it works

- **The state is a `CartResponse`.** `state.cart.response` looks exactly like the body of `GET /cart`. In Part 2, storing a server response is one line: `state.response = action.payload`.
- `emptyCart()` is an empty cart in the server's shape, so selectors never need null checks.
- `recalculate` copies the server's rules: `numOfCartItems` is the number of *different* products (your real cart shows 6 even though some lines have a count of 11), and `totalCartPrice` is the sum of `count × price`.
- **"Mutating" inside reducers is safe.** RTK uses **Immer**: you write `existing.count += 1`, and Immer produces a new immutable state behind the scenes. It's like `copyWith`, without writing it.
- `PayloadAction<ProductDto>` types `action.payload`, and therefore the argument of `addToCart(…)` everywhere it's dispatched.
- `export const {addToCart, …} = cartSlice.actions` uses destructuring: `createSlice` generated one **action creator** per reducer. `addToCart(p)` returns `{type: "cart/addToCart", payload: p}`.
- **Selectors** are the only code that knows the state's shape. Screens call `selectCount(state, id)` and never `state.cart.response.data.products…`.

1.5 · The store and typed hooks

> src/store/store.ts

```
import {configureStore} from "@reduxjs/toolkit";
import cartReducer from "./cart/cartSlice.ts";

export const store = configureStore({
    reducer: {
        cart: cartReducer,   // state.cart
    },
});

// Types inferred from the store itself, so they update when you add slices
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

> src/store/hooks.ts

```
import {useDispatch, useSelector} from "react-redux";
import type {AppDispatch, RootState} from "./store.ts";

// Typed versions of the react-redux hooks: use these instead of the plain ones
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

- `RootState` is the type of the whole state (`{cart: CartState}`), worked out from the store. Add a slice and it updates automatically.
- `AppDispatch` is the store's real `dispatch` type. It also knows about thunks, which you'll need in Part 2.
- `useAppSelector` / `useAppDispatch` are react-redux's hooks pre-typed with those types, so `state` autocompletes without writing `: RootState` everywhere.

1.6 · Provide the store (App.tsx)

Your `<>…</>` Fragment becomes `<Provider>`. It still groups the navigator and the `Toast`, and now every component inside can reach the store:

> App.tsx

```
/**
 * Sample React Native App
 * https://github.com/facebook/react-native
 *
 * @format
 */
import {StatusBar, useColorScheme} from 'react-native';
import {
    SafeAreaProvider,
} from 'react-native-safe-area-context';
```

```
import HomeTab from "./src/screens/main_tabs/tabs/home_tab/HomeTab.tsx";
import {GestureHandlerRootView} from "react-native-gesture-handler";
import {NavigationContainer} from "@react-navigation/native";
import RootNavigation from "./src/screens/RootNavigation.tsx";
import Toast from "react-native-toast-message";
import {Provider} from "react-redux";
import {store} from "./src/store/store.ts";

function App() {
    const isDarkMode = useColorScheme() === 'dark';
    return (
        <Provider store={store}>
            {<NavigationContainer>
                <GestureHandlerRootView>
                    <SafeAreaProvider>
                        {/*    <StatusBar barStyle={isDarkMode ? 'light-content' : 'dark-
content'}/>*/}
                        {/*    <HomeTab/>*/}
                        <RootNavigation/>
                    </SafeAreaProvider>
                </GestureHandlerRootView>
            </NavigationContainer>}
            <Toast position={"bottom"}/>
        </Provider>
    );
}

export default App;
```

1.7 · The quantity control and the add button

`QuantityControl` only draws **– count +**. It doesn't know about Redux, so both the product card and the cart screen can reuse it. The `loading` prop is used in Part 2.

> src/common/QuantityControl.tsx

```
import {ActivityIndicator, Pressable, StyleSheet, Text, View} from "react-native";
import {colors} from "../theme/colors.ts";

type QuantityControlProps = {
    count: number;
    onIncrease: () => void;
    onDecrease: () => void;
    loading?: boolean;   // used once the buttons call the API
};

// Only draws – count +. It doesn't know about Redux: the parent decides what the buttons do.
export default function QuantityControl({count, onIncrease, onDecrease, loading = false}:
QuantityControlProps) {
```

```
    return (
        <View style={styles.row}>
            <Pressable style={styles.circle} onPress={onDecrease} disabled={loading}>
                <Text style={styles.symbol}>−</Text>
            </Pressable>
            {loading
                ? <ActivityIndicator size="small" color={colors.primary} style={styles.count}/>
                : <Text style={styles.count}>{count}</Text>}
            <Pressable style={styles.circle} onPress={onIncrease} disabled={loading}>
                <Text style={styles.symbol}>+</Text>
            </Pressable>
        </View>
    );
}

export const quantityStyles = StyleSheet.create({
    circle: {
        width: 30,
        height: 30,
        borderRadius: 15,
        backgroundColor: colors.primary,
        justifyContent: "center",
        alignItems: "center",
    },
    symbol: {color: colors.white, fontSize: 18, lineHeight: 20},
});

const styles = StyleSheet.create({
    ...quantityStyles,
    row: {flexDirection: "row", alignItems: "center", gap: 8},
    count: {color: colors.primary, fontSize: 16, fontWeight: "600", minWidth: 20, textAlign:
"center"},
});
```

src/common/AddToCartButton.tsx

```
import {Pressable, Text} from "react-native";
import {ProductDto} from "../api/models/response/productResponse.ts";
import {useAppDispatch, useAppSelector} from "../store/hooks.ts";
import {addToCart, selectCount, updateCount} from "../store/cart/cartSlice.ts";
import QuantityControl, {quantityStyles} from "./QuantityControl.tsx";

export default function AddToCartButton({product}: { product: ProductDto }) {
    const dispatch = useAppDispatch();
    // Re-renders only when THIS product's count changes
    const count = useAppSelector(state => selectCount(state, product._id));

    // Not in the cart yet: the round + button
    if (count === 0) {
```

```
        return (
            <Pressable style={quantityStyles.circle} onPress={() => dispatch(addToCart(product))}>
                <Text style={quantityStyles.symbol}>+</Text>
            </Pressable>
        );
    }

    // In the cart: − count +
    return (
        <QuantityControl
            count={count}
            onIncrease={() => dispatch(updateCount({productId: product._id, count: count + 1}))}
            onDecrease={() => dispatch(updateCount({productId: product._id, count: count - 1}))}
        />
    );
}
```

- `useAppSelector(state => selectCount(state, product._id))` reads this product's count and subscribes to it. After any dispatch it re-runs, and the button re-renders only if *its* count changed. Tapping one product doesn't re-render the others.
- **Count 0** shows the round **+**. Anything else shows **− count +**. Going from 1 to 0 removes the line in the reducer, so the **+** comes back by itself.

1.8 · The cart screen and its route

> src/screens/cart/CartScreen.tsx

```
import {FlatList, Image, Pressable, StyleSheet, Text, View} from "react-native";
import {useAppDispatch, useAppSelector} from "../../store/hooks.ts";
import {removeFromCart, selectCartItems, selectCartTotal, updateCount} from
"../../store/cart/cartSlice.ts";
import {CartItemDto} from "../../api/models/response/cartResponse.ts";
import QuantityControl from "../../common/QuantityControl.tsx";
import {colors} from "../../theme/colors.ts";

export default function CartScreen() {
    const items = useAppSelector(selectCartItems);
    const total = useAppSelector(selectCartTotal);

    return (
        <View style={styles.screen}>
            <FlatList
                data={items}
                keyExtractor={item => item._id}
                contentContainerStyle={styles.list}
                ListEmptyComponent={<Text style={styles.empty}>Your cart is empty</Text>}
                renderItem={({item}) => <CartLine item={item}/>}
            />
            <View style={styles.footer}>
```

```
                <Text style={styles.totalLabel}>Total price</Text>
                <Text style={styles.total}>EGP {total}</Text>
            </View>
        </View>
    );
}

function CartLine({item}: { item: CartItemDto }) {
    const dispatch = useAppDispatch();
    const productId = item.product._id;

    return (
        <View style={styles.line}>
            <Image source={{uri: item.product.imageCover}} style={styles.image}/>
            <View style={styles.info}>
                <Text style={styles.title} numberOfLines={2}>{item.product.title}</Text>
                <Text style={styles.price}>EGP {item.price}</Text>
                <View style={styles.actions}>
                    <QuantityControl
                        count={item.count}
                        onIncrease={() => dispatch(updateCount({productId, count: item.count + 1}))}
                        onDecrease={() => dispatch(updateCount({productId, count: item.count - 1}))}
                    />
                    <Pressable onPress={() => dispatch(removeFromCart(productId))}>
                        <Text style={styles.remove}>Remove</Text>
                    </Pressable>
                </View>
            </View>
        </View>
    );
}

const styles = StyleSheet.create({
    screen: {flex: 1, backgroundColor: colors.white},
    list: {padding: 16, gap: 12},
    empty: {textAlign: "center", marginTop: 48, color: colors.primary, fontSize: 16},
    line: {flexDirection: "row", gap: 12, borderWidth: 1, borderColor: colors.border, borderRadius:
16, padding: 8},
    image: {width: 90, height: 90, borderRadius: 12},
    info: {flex: 1, gap: 6},
    title: {color: colors.primary, fontWeight: "600"},
    price: {color: colors.primary},
    actions: {flexDirection: "row", justifyContent: "space-between", alignItems: "center"},
    remove: {color: colors.danger, fontWeight: "600"},
    footer: {
        flexDirection: "row",
        justifyContent: "space-between",
        padding: 16,
        borderTopWidth: 1,
```

```
        borderColor: colors.border,
    },
    totalLabel: {color: colors.primary, fontSize: 16},
    total: {color: colors.primary, fontSize: 18, fontWeight: "bold"},
});
```

Register it in the stack, with the header on, so it gets a back button:

src/screens/RootNavigation.tsx

```
import {createNativeStackNavigator} from "@react-navigation/native-stack";
import LoginScreen from "./login/LoginScreen.tsx";
import MainTabs from "./main_tabs/MainTabs.tsx";
import CartScreen from "./cart/CartScreen.tsx";
import {useEffect, useState} from "react";
import {tokenStorage} from "../core/async_storage/asyncStorageHelper.ts";
import {ActivityIndicator, View} from "react-native";


export type RootStackParams = {
    Login: undefined;
    MainTabs: undefined;
    Cart: undefined;
}
const Stack = createNativeStackNavigator<RootStackParams>()


export default function RootNavigation() {
    const [initialRoute, setInitialRouteName] = useState<keyof RootStackParams | null>(null);
    useEffect(() => {
        tokenStorage.get().then((token) => {
            console.log(`Token = ${token}`)
            if (token) {
                console.log(`Setting route to main`)
                setInitialRouteName('MainTabs');
            } else {
                console.log(`Setting route to login`)
                setInitialRouteName('Login');
            }
        }).catch((error) => {
            console.log(`error = ${error}`)
            setInitialRouteName('Login');
        })
    }, [])

    console.log(`35- RootNavigation ${initialRoute}`)
    if (!initialRoute) {
        return (
            <View>
```

```
                <ActivityIndicator/>
            </View>
        );

    }

    return (<Stack.Navigator initialRouteName={initialRoute!} screenOptions={{headerShown: false}}>
        <Stack.Screen name={"Login"} component={LoginScreen}/>
        <Stack.Screen name={"MainTabs"} component={MainTabs}/>
        <Stack.Screen name={"Cart"} component={CartScreen} options={{headerShown: true, title:
"Cart"}}/>

    </Stack.Navigator>)
}
```

1.9 · HomeTab: the add button and the badge

Here's HomeTab from step 1 with three changes:

- **Imports:** `Pressable`, navigation (`useNavigation`, `NativeStackNavigationProp`, `RootStackParams`), `AddToCartButton`, `useAppSelector` and `selectCartCount`.
- **`ProductItem`:** the `+` `Text` becomes `<AddToCartButton product={product}/>`. `alignContent` becomes `alignItems`, so the review text stays centered next to the wider control.
- **`CartIcon`:** in `SearchBarAndCartRow`, the cart image becomes `<CartIcon/>`, which shows a red badge (`numOfCartItems`) and opens the cart screen. Two styles are added: `badge` and `badgeText`.

src/screens/main_tabs/tabs/home_tab/HomeTab.tsx

```
import {useEffect, useState} from "react";
import {ActivityIndicator, FlatList, Image, Pressable, ScrollView, StyleSheet, Text, TextInput,
useWindowDimensions, View} from "react-native";
import Toast from "react-native-toast-message";
import {getCategories, getProducts} from "../../../../api/apiServices.ts";
import {getErrorMessage} from "../../../../api/errors.ts";
import {CategoryDto} from "../../../../api/models/response/categoryResponse.ts";
import {ProductDto} from "../../../../api/models/response/productResponse.ts";
import {useNavigation} from "@react-navigation/native";
import {NativeStackNavigationProp} from "@react-navigation/native-stack";
import type {RootStackParams} from "../../../RootNavigation.tsx";
import AddToCartButton from "../../../../common/AddToCartButton.tsx";
import {useAppSelector} from "../../../../store/hooks.ts";
import {selectCartCount} from "../../../../store/cart/cartSlice.ts";
import {Carousel} from "react-native-reanimated-carousel";
import {SafeAreaView} from "react-native-safe-area-context";

// Text -> Text
// Button - Button
// Images - Image
// Column - Row - Stack -> View
// ListView.builder -> Flatlist
```

```
// SingleChildScrollView -> ScrollView
//TextField -> TextInput
export const blue = '#004182'

function HomeTab() {
    return (
        <SafeAreaView edges={['top']}>
            <ScrollView>
                <View style={styles.screenContainer}>
                    <Image source={require("../../../../assets/images/route_logo.png")}
                           style={styles.logo}/>
                    <SearchBarAndCartRow/>
                    <AdsCarosuel/>
                    <Categories/>
                    <ProductsSection/>
                </View>
            </ScrollView>
        </SafeAreaView>

      )
}

function SearchBarAndCartRow() {
    return (<View style={{flexDirection: 'row', alignItems: 'center', gap: 8}}>
        <View style={styles.searchContainer}>
            <Image source={require("../../../../assets/images/ic_search.png")}/>

            <TextInput placeholder='What do u search for?'
                       placeholderTextColor={blue} style={{color: blue}}/>
        </View>
        <CartIcon/>
    </View>)
}

function CartIcon() {
    const navigation = useNavigation<NativeStackNavigationProp<RootStackParams>>();
    // Re-renders when the number of products in the cart changes
    const count = useAppSelector(selectCartCount);
    return (<Pressable onPress={() => navigation.navigate("Cart")}>
        <Image source={require("../../../../assets/images/ic_cart.png")}/>
        {count > 0 ? (
            <View style={styles.badge}>
                <Text style={styles.badgeText}>{count}</Text>
            </View>
        ) : null}
    </Pressable>)
}

const ads = [
```

```
    require("../../../../assets/images/ads_img.png"),
    require("../../../../assets/images/ads_img.png"),
    require("../../../../assets/images/ads_img.png"),
]

function AdsCarosuel() {
    const {width} = useWindowDimensions();
    const fullWidth = width - 32
    return (<Carousel
        style={{height: 200, width: fullWidth}}
        data={ads}
        autoplay={true}
        autoplayInterval={2000}
        renderItem={({item}) => (
            <View style={{alignItems: "center", justifyContent: "center", }}>
                <Image style={{marginHorizontal: 16, padding:16, width: fullWidth - 16}} source=
{item}/>
            </View>
        )}
    />)
}

function Categories() {
    const [categories, setCategories] = useState<CategoryDto[]>([]);
    const [loading, setLoading] = useState(true);

    // Runs once when the section appears, like initState
    useEffect(() => {
        getCategories()
            .then(setCategories)
            .catch(error => Toast.show({type: "error", text1: "Categories", text2:
getErrorMessage(error)}))
            .finally(() => setLoading(false));
    }, []);

    // [a, b, c, d, e] → [[a, b], [c, d], [e]] for the two-row horizontal list
    const categoryPairs: CategoryDto[][] = [];
    for (let i = 0; i < categories.length; i += 2) {
        categoryPairs.push(categories.slice(i, i + 2))
    }

    return (<View>
        <View style={{flexDirection: 'row', justifyContent: 'space-between', gap: 8}}>
            <Text style={{color: blue}}>Categories</Text>
            <Text style={{color: blue}}>View all</Text>
        </View>
        {loading ? <ActivityIndicator color={blue} style={{marginTop: 16}}/> : (
            <FlatList
                horizontal={true}
```

```
                data={categoryPairs}
                showsHorizontalScrollIndicator={false}
                renderItem={({item}) => <View style={{gap: 8}}>
                    {item.map((category) => <CategoryItem key={category._id} category={category}/>)}
                </View>}
                keyExtractor={(pair) => pair[0]._id}
            />
        )}
    </View>)
}

function CategoryItem({category}: { category: CategoryDto }) {
    return (<View style={{marginHorizontal: 16, width: 100, alignItems: 'center'}}>
        <Image source={{uri: category.image}} style={{width: 100, height: 100, borderRadius: 50}}/>
        <Text style={{color: blue, textAlign: 'center'}} numberOfLines={2}>{category.name}</Text>
    </View>)
}

function ProductsSection() {
    const [products, setProducts] = useState<ProductDto[]>([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        getProducts(10)
            .then(setProducts)
            .catch(error => Toast.show({type: "error", text1: "Products", text2:
getErrorMessage(error)}))
            .finally(() => setLoading(false));
    }, []);

    return (<View>
        <View style={{flexDirection: 'row', justifyContent: 'space-between', gap: 8}}>
            <Text style={{color: blue}}>Products</Text>
            <Text style={{color: blue}}>View all</Text>
        </View>
        {loading ? <ActivityIndicator color={blue} style={{marginTop: 16}}/> : (
            <FlatList
                horizontal={true}
                data={products}
                showsHorizontalScrollIndicator={false}
                renderItem={({item}) => <ProductItem product={item}/>}
                keyExtractor={(item) => item._id}
            />
        )}
    </View>)
}

function ProductItem({product}: { product: ProductDto }) {
    return (<View style={{
```

```
        borderRadius: 16,
        // borderColor: '#0041824D',
        width: 190,
        marginHorizontal: 8,
        marginTop: 16,
        borderWidth: 1,
        paddingBottom: 8
    }}>
        <Image source={{uri: product.imageCover}}
               style={{width: '100%', height: 120, borderTopLeftRadius: 16, borderTopRightRadius:
16}}/>
        <View style={{marginHorizontal: 8, gap: 8}}>
            <Text style={{color: blue,}} numberOfLines={2}>{product.title}</Text>
            <Text style={{color: blue}}>EGP {product.price}</Text>
            <View style={{flexDirection: 'row', justifyContent: 'space-between', alignItems:
'center'}}>
                <Text style={{color: blue, verticalAlign: "middle"}}>Review
({product.ratingsAverage})</Text>
                <AddToCartButton product={product}/>
            </View>
        </View>

        <Image style={{position: 'absolute', right: 0}} source=
{require("../../../../assets/images/ic_favorite.png")}/>

    </View>)
}

const styles = StyleSheet.create({
    screenContainer: {
        marginHorizontal: 16,
        flex: 1,
        gap: 18
    },
    searchContainer: {
        flexDirection: 'row',
        borderColor: blue,
        borderWidth: 1,
        borderRadius: 24,
        paddingHorizontal: 18,
        paddingVertical: 8,
        alignItems: 'center',
        gap: 4,
        flex: 1
    },
    logo: {
        marginTop: 12
    },
    badge: {
```

```
        position: 'absolute',
        top: -6,
        right: -8,
        minWidth: 18,
        height: 18,
        borderRadius: 9,
        paddingHorizontal: 4,
        backgroundColor: 'red',
        justifyContent: 'center',
        alignItems: 'center',
    },
    badgeText: {color: 'white', fontSize: 11, fontWeight: 'bold'},
})
export default HomeTab;
```

Part 1 checklist

☐ Step 1 done ☐ `yarn add @reduxjs/toolkit react-redux` ☐ `colors.ts` ☐ `cartResponse.ts` ☐ `cartSlice.ts` + `store.ts` + `hooks.ts` ☐ `<Provider>` in App.tsx ☐ `QuantityControl` + `AddToCartButton` ☐ `CartScreen` + the `Cart` route ☐ HomeTab changes

Try it: tap + on two products, the badge shows 2. Tap + again on one, the badge still shows 2 (different products), but the count goes to 2. Open the cart and check the total. Press – to 0, and the line disappears and the product card shows + again.

Part 2 · Switch to the API

2.1 · What changes and what doesn't

Stays exactly the same	Changes
• The models (`ProductDto` , `CartResponse` …) and `errors.ts` • `store.ts` , `hooks.ts` , `App.tsx` , `RootNavigation.tsx` , `HomeTab.tsx` • `QuantityControl` • Action names and arguments: `addToCart(product)` , `updateCount({productId, count})` , `removeFromCart(id)` , `clearCart()` • The selectors `selectCount` , `selectCartCount` , `selectCartTotal` , `selectCartItems`	• `apiServices.ts` : export the client, send the token • New: `cartService.ts` • `cartSlice.ts` : local reducers → thunks • Button and cart screen: a spinner and an error toast • `MainTabs` : load the cart. `ProfileTab` : logout

2.2 · Send the token (apiServices.ts)

Your `apiServices.ts` saves the token at login, but its request interceptor only logs, so the `token` header is never sent. I confirmed that `GET /cart` without it answers: "*You are not logged in. Please login to get access*". Make two changes:

1. **Export the client:** `const apiClient` → `export const apiClient` , so the new services can use it.
2. **Attach the token** at the top of the existing request interceptor. The interceptor can be `async` , and axios waits for it:

src/api/apiServices.ts

```
// var dio = Dio(BaseOptions(baseUrl: "", headers))

import axios, {AxiosError, AxiosResponse, InternalAxiosRequestConfig} from "axios";
import {AuthResponse, User} from "./models/response/authResponse.ts";
import {tokenStorage} from "../core/async_storage/asyncStorageHelper.ts";
import {CategoriesResponse, CategoryDto} from "./models/response/categoryResponse.ts";
import {ProductDto, ProductsResponse} from "./models/response/productResponse.ts";

export const apiClient = axios.create({
    baseURL: 'https://ecommerce.routemisr.com/api/v1/',
    headers: {
        'Content-Type': 'application/json',
    }
})


export async function login(email: string, password: string): Promise<any> {
    const response = await apiClient.post<AuthResponse>("auth/signin", {email, password});
    if (response.data.token) {
        await tokenStorage.save(response.data.token);
    }
```

```
    return response.data;
}

export async function getCategories(): Promise<CategoryDto[]> {
    const response = await apiClient.get<CategoriesResponse>("categories");
    return response.data.data;   // axios body → the API's `data` array
}

// limit = how many products to load (the API's default is 40)
export async function getProducts(limit?: number): Promise<ProductDto[]> {
    const response = await apiClient.get<ProductsResponse>("products", {params: {limit}});
    return response.data.data;
}


// 1. REQUEST INTERCEPTOR
apiClient.interceptors.request.use(
    async (config: InternalAxiosRequestConfig) => {
        // Route reads the login token from a `token` header
        const token = await tokenStorage.get();
        if (token) {
            config.headers.token = token;
        }

        const method = config.method?.toUpperCase() || 'METHOD';

        console.group(`🚀 [API Request] ${method} → ${config.url}`);

        // Log headers safely (avoids [object Object])
        if (config.headers) {
            console.log('Headers:', JSON.parse(JSON.stringify(config.headers)));
        }

        // Log request body/params if they exist
        if (config.data) {
            console.log('Body:', config.data);
        }
        if (config.params) {
            console.log('Params:', config.params);
        }

        console.groupEnd();
        return config;
    },
    (error: AxiosError) => {
        console.error('❌ [API Request Error]', error);
        return Promise.reject(error);
    }
);
```

```
// 2. RESPONSE INTERCEPTOR
apiClient.interceptors.response.use(
    (response: AxiosResponse) => {
        console.group(`✅ [API Response] ${response.status} → ${response.config.url}`);

        // Log the actual server payload data
        if (response.data) {
            console.log('Data:', response.data);
        }

        console.groupEnd();
        return response;
    },
    (error: AxiosError) => {
        // Gracefully handle server responses with error codes (400, 401, 500, etc.)
        if (error.response) {
            console.group(`❌ [API Error Response] ${error.response.status} →
${error.config?.url}`);
            console.log('Error Data:', error.response.data);
            console.groupEnd();
        } else if (error.request) {
            // The request was made but no response was received (e.g., Network Down)
            console.error('📡 [API Network Error] No response received:', error.request);
        } else {
            console.error('⚙️ [API Setup Error]:', error.message);
        }

        return Promise.reject(error);
    }
);
```

2.3 · The cart service, including the temporary GET

> src/api/cartService.ts

```
import {apiClient} from "./apiServices.ts";
import {CartResponse} from "./models/response/cartResponse.ts";

export async function getCart(): Promise<CartResponse> {
    const response = await apiClient.get<CartResponse>("cart");
    return response.data;
}

export async function addProductToCart(productId: string): Promise<CartResponse> {
    await apiClient.post("cart", {productId});
    // TEMPORARY: POST /cart returns `product` as an id string today.
    // Until the backend returns the full shape, load the cart again.
    // When it does, replace both lines with:
    //   const response = await apiClient.post<CartResponse>("cart", {productId});
    //   return response.data;
    return getCart();
}

export async function updateCartCount(productId: string, count: number): Promise<CartResponse> {
    const response = await apiClient.put<CartResponse>(`cart/${productId}`, {count});
    return response.data;
}

export async function removeProductFromCart(productId: string): Promise<CartResponse> {
    const response = await apiClient.delete<CartResponse>(`cart/${productId}`);
    return response.data;
}
```

The temporary part: `addProductToCart` ignores the POST body (its `product` is still an id today) and returns a fresh `GET /cart` instead. **When the backend ships the full shape**, replace those two lines with the commented version. Nothing else in the app changes, because the function's return type is already `CartResponse`.

Check in Postman first: the `:productId` in the PUT and DELETE URLs is the **product's** `_id`, not the cart line's `_id`. That's how Route's cart API is documented. Try it once with your token before relying on it.

2.4 · The cart slice (API)

This replaces the whole Part 1 file. The action names, their arguments and the selectors are the same, so the screens keep working:

src/store/cart/cartSlice.ts (Part 2)

```
import {createAsyncThunk, createSlice, isAnyOf} from "@reduxjs/toolkit";
import {ProductDto} from "../../api/models/response/productResponse.ts";
import {CartResponse} from "../../api/models/response/cartResponse.ts";
import {addProductToCart, getCart, removeProductFromCart, updateCartCount} from
"../../api/cartService.ts";
import {getErrorMessage} from "../../api/errors.ts";
```

```ts
import type {RootState} from "../store.ts";

interface CartState {
    response: CartResponse;                // exactly what the server returned last
    loading: boolean;                      // fetchCart in progress (cart screen spinner)
    updatingIds: Record<string, boolean>;  // productId → its button is waiting for the server
    error: string | null;
}

// An empty cart in the server's shape
function emptyCart(): CartResponse {
    return {
        status: "success",
        numOfCartItems: 0,
        cartId: "",
        data: {_id: "", cartOwner: "", products: [], totalCartPrice: 0},
    };
}

const initialState: CartState = {
    response: emptyCart(),
    loading: false,
    updatingIds: {},
    error: null,
};

// createAsyncThunk pre-typed for this app: errors come back as a string message
const createCartThunk = createAsyncThunk.withTypes<{ state: RootState; rejectValue: string }>();

// ---- thunks: same names and arguments as the local actions, so screens don't change ----

export const fetchCart = createCartThunk("cart/fetch", async (_: void, {rejectWithValue}) => {
    try {
        return await getCart();
    } catch (error) {
        return rejectWithValue(getErrorMessage(error));
    }
});

export const addToCart = createCartThunk("cart/add", async (product: ProductDto, {rejectWithValue})
=> {
    try {
        return await addProductToCart(product._id);
    } catch (error) {
        return rejectWithValue(getErrorMessage(error));
    }
});

export const updateCount = createCartThunk(
```

```
    "cart/updateCount",
    async ({productId, count}: { productId: string; count: number }, {rejectWithValue}) => {
        try {
            // going to 0 removes the line, so the + button comes back
            return count > 0 ? await updateCartCount(productId, count) : await
removeProductFromCart(productId);
        } catch (error) {
            return rejectWithValue(getErrorMessage(error));
        }
    },
);

export const removeFromCart = createCartThunk("cart/remove", async (productId: string,
{rejectWithValue}) => {
    try {
        return await removeProductFromCart(productId);
    } catch (error) {
        return rejectWithValue(getErrorMessage(error));
    }
});

// Which product a thunk is about, from the argument it was called with
function productIdOf(arg: ProductDto | string | { productId: string }): string {
    if (typeof arg === "string") return arg;
    return "productId" in arg ? arg.productId : arg._id;
}

const cartSlice = createSlice({
    name: "cart",
    initialState,
    reducers: {
        clearCart() {
            return initialState;   // on logout: forget the previous user's cart
        },
    },
    extraReducers: builder => {
        builder
            .addCase(fetchCart.pending, state => {
                state.loading = true;
            })
            .addCase(fetchCart.rejected, state => {
                state.loading = false;
            });

        // Every cart API answers with the same CartResponse, so one handler stores them all
        builder.addMatcher(
            isAnyOf(fetchCart.fulfilled, addToCart.fulfilled, updateCount.fulfilled,
removeFromCart.fulfilled),
            (state, action) => {
```

```
                state.response = action.payload;
                state.loading = false;
                state.error = null;
            },
        );

        // Per-product spinner while its request runs
        builder.addMatcher(
            isAnyOf(addToCart.pending, updateCount.pending, removeFromCart.pending),
            (state, action) => {
                state.updatingIds[productIdOf(action.meta.arg)] = true;
            },
        );
        builder.addMatcher(
            isAnyOf(
                addToCart.fulfilled, updateCount.fulfilled, removeFromCart.fulfilled,
                addToCart.rejected, updateCount.rejected, removeFromCart.rejected,
            ),
            (state, action) => {
                delete state.updatingIds[productIdOf(action.meta.arg)];
            },
        );

        builder.addMatcher(
            isAnyOf(fetchCart.rejected, addToCart.rejected, updateCount.rejected,
removeFromCart.rejected),
            (state, action) => {
                state.error = action.payload ?? "Something went wrong";
            },
        );
    },
});

export const {clearCart} = cartSlice.actions;
export default cartSlice.reducer;

// ---- selectors: unchanged from the local version, plus two for loading ----
export const selectCartItems = (state: RootState) => state.cart.response.data.products;
export const selectCartCount = (state: RootState) => state.cart.response.numOfCartItems;
export const selectCartTotal = (state: RootState) => state.cart.response.data.totalCartPrice;
export const selectCount = (state: RootState, productId: string) =>
    state.cart.response.data.products.find(item => item.product._id === productId)?.count ?? 0;
export const selectIsUpdating = (state: RootState, productId: string) =>
state.cart.updatingIds[productId] ?? false;
export const selectCartLoading = (state: RootState) => state.cart.loading;
```

New concepts, one by one

- `createAsyncThunk("cart/add", async (product) => …)` creates an async action. Dispatching it runs your function, and Redux automatically dispatches three actions around it:

Action	When	What the slice does
`cart/add/pending`	Right before the call	Marks the product as updating (spinner)
`cart/add/fulfilled`	The function returned; `action.payload` = the `CartResponse`	`state.response = action.payload`
`cart/add/rejected`	The function threw or called `rejectWithValue`	Stores the error, clears the spinner

- **`createAsyncThunk.withTypes<{state: RootState; rejectValue: string}>()`** gives a pre-typed version, like `useAppSelector`: the error value is always a `string`.
- **`rejectWithValue(getErrorMessage(error))`** rejects with *our* message instead of a raw axios error. That's what the button's toast shows.
- **`extraReducers`** handles actions the slice didn't create itself, such as the thunks' pending/fulfilled/rejected. `reducers` is only for the slice's own synchronous actions, like `clearCart`.
- **`addCase(action, handler)`** handles one action. **`addMatcher(isAnyOf(a, b, c), handler)`** handles several with one handler, which is why one line stores the response of all four endpoints. That only works because the backend returns the same shape everywhere.
- **`action.meta.arg`** is the argument the thunk was called with (the product, the id, or `{productId, count}`). `productIdOf` turns any of them into the id, so the spinner knows which button to show.
- **`recalculate` is gone:** the server sends the totals.

2.5 · The button: spinner and error toast

> src/common/AddToCartButton.tsx (Part 2)

```
import {ActivityIndicator, Pressable, Text} from "react-native";
import Toast from "react-native-toast-message";
import {ProductDto} from "../api/models/response/productResponse.ts";
import {useAppDispatch, useAppSelector} from "../store/hooks.ts";
import {addToCart, selectCount, selectIsUpdating, updateCount} from "../store/cart/cartSlice.ts";
import QuantityControl, {quantityStyles} from "./QuantityControl.tsx";
import {colors} from "../theme/colors.ts";

// Shows the server's message when a cart request fails.
// unwrap() turns the thunk's result back into a normal promise that rejects on failure.
function showErrorToast(message: string) {
    Toast.show({type: "error", text1: "Cart", text2: message});
}

export default function AddToCartButton({product}: { product: ProductDto }) {
    const dispatch = useAppDispatch();
    const count = useAppSelector(state => selectCount(state, product._id));
    const updating = useAppSelector(state => selectIsUpdating(state, product._id));

    if (count === 0) {
        return (
            <Pressable style={quantityStyles.circle} disabled={updating}
                       onPress={() => dispatch(addToCart(product)).unwrap().catch(showErrorToast)}>
```

```
                {updating
                    ? <ActivityIndicator size="small" color={colors.white}/>
                    : <Text style={quantityStyles.symbol}>+</Text>}
            </Pressable>
        );
    }

    return (
        <QuantityControl
            count={count}
            loading={updating}
            onIncrease={() => dispatch(updateCount({productId: product._id, count: count +
1})).unwrap().catch(showErrorToast)}
            onDecrease={() => dispatch(updateCount({productId: product._id, count: count -
1})).unwrap().catch(showErrorToast)}
        />
    );
}
```

- `dispatch(thunk(...)).unwrap()` : dispatching a thunk returns a promise of its *result action*, which never throws. `.unwrap()` turns it back into a normal promise that rejects with the `rejectValue`, the message. So `.catch(showErrorToast)` receives the server's text.
- **Toasts belong in components, not reducers.** A reducer must only calculate the new state, with no side effects. The component that started the action is the right place to show feedback.
- `disabled={updating}` stops double taps while a request is running.

2.6 · The cart screen: load on open, pull to refresh

> src/screens/cart/CartScreen.tsx (Part 2)

```
import {useEffect} from "react";
import {ActivityIndicator, FlatList, Image, Pressable, StyleSheet, Text, View} from "react-native";
import Toast from "react-native-toast-message";
import {useAppDispatch, useAppSelector} from "../../store/hooks.ts";
import {
    fetchCart,
    removeFromCart,
    selectCartItems,
    selectCartLoading,
    selectCartTotal,
    selectIsUpdating,
    updateCount,
} from "../../store/cart/cartSlice.ts";
import {CartItemDto} from "../../api/models/response/cartResponse.ts";
import QuantityControl from "../../common/QuantityControl.tsx";
import {colors} from "../../theme/colors.ts";

function showErrorToast(message: string) {
    Toast.show({type: "error", text1: "Cart", text2: message});
```

```
}

export default function CartScreen() {
    const dispatch = useAppDispatch();
    const items = useAppSelector(selectCartItems);
    const total = useAppSelector(selectCartTotal);
    const loading = useAppSelector(selectCartLoading);

    // Always show the server's latest cart when the screen opens
    useEffect(() => {
        dispatch(fetchCart()).unwrap().catch(showErrorToast);
    }, [dispatch]);

    if (loading && items.length === 0) {
        return <View style={styles.center}><ActivityIndicator size="large" color={colors.primary}/>
</View>;
    }

    return (
        <View style={styles.screen}>
            <FlatList
                data={items}
                keyExtractor={item => item._id}
                contentContainerStyle={styles.list}
                refreshing={loading}
                onRefresh={() => dispatch(fetchCart()).unwrap().catch(showErrorToast)}   // pull to
refresh
                ListEmptyComponent={<Text style={styles.empty}>Your cart is empty</Text>}
                renderItem={({item}) => <CartLine item={item}/>}
            />
            <View style={styles.footer}>
                <Text style={styles.totalLabel}>Total price</Text>
                <Text style={styles.total}>EGP {total}</Text>
            </View>
        </View>
    );
}

function CartLine({item}: { item: CartItemDto }) {
    const dispatch = useAppDispatch();
    const productId = item.product._id;
    const updating = useAppSelector(state => selectIsUpdating(state, productId));

    return (
        <View style={styles.line}>
            <Image source={{uri: item.product.imageCover}} style={styles.image}/>
            <View style={styles.info}>
                <Text style={styles.title} numberOfLines={2}>{item.product.title}</Text>
                <Text style={styles.price}>EGP {item.price}</Text>
```

```
                <View style={styles.actions}>
                    <QuantityControl
                        count={item.count}
                        loading={updating}
                        onIncrease={() => dispatch(updateCount({productId, count: item.count +
1})).unwrap().catch(showErrorToast)}
                        onDecrease={() => dispatch(updateCount({productId, count: item.count -
1})).unwrap().catch(showErrorToast)}
                    />
                    <Pressable disabled={updating}
                               onPress={() =>
dispatch(removeFromCart(productId)).unwrap().catch(showErrorToast)}>
                        <Text style={styles.remove}>Remove</Text>
                    </Pressable>
                </View>
            </View>
        </View>
    );
}

const styles = StyleSheet.create({
    screen: {flex: 1, backgroundColor: colors.white},
    center: {flex: 1, justifyContent: "center", alignItems: "center", backgroundColor:
colors.white},
    list: {padding: 16, gap: 12},
    empty: {textAlign: "center", marginTop: 48, color: colors.primary, fontSize: 16},
    line: {flexDirection: "row", gap: 12, borderWidth: 1, borderColor: colors.border, borderRadius:
16, padding: 8},
    image: {width: 90, height: 90, borderRadius: 12},
    info: {flex: 1, gap: 6},
    title: {color: colors.primary, fontWeight: "600"},
    price: {color: colors.primary},
    actions: {flexDirection: "row", justifyContent: "space-between", alignItems: "center"},
    remove: {color: colors.danger, fontWeight: "600"},
    footer: {
        flexDirection: "row",
        justifyContent: "space-between",
        padding: 16,
        borderTopWidth: 1,
        borderColor: colors.border,
    },
    totalLabel: {color: colors.primary, fontSize: 16},
    total: {color: colors.primary, fontSize: 18, fontWeight: "bold"},
});
```

2.7 · Load the cart after login (MainTabs.tsx)

`MainTabs` mounts after login and on app start when a token is saved. Loading the cart there makes the badge and the product buttons show what's already in the server cart:

src/screens/main_tabs/MainTabs.tsx

```
import {useEffect} from "react";
import {Image, ImageSourcePropType, View} from "react-native";
import {useAppDispatch} from "../../store/hooks.ts";
import {fetchCart} from "../../store/cart/cartSlice.ts";
import {createBottomTabNavigator} from "@react-navigation/bottom-tabs";
import HomeTab, {blue} from "./tabs/home_tab/HomeTab.tsx";
import CategoriesTab from "./tabs/categories_tab/CategoriesTab.tsx";
import ProfileTab from "./tabs/profile_tab/ProfileTab.tsx";
import WishListTab from "./tabs/wish_list_tab/WishListTab.tsx";

const Tab = createBottomTabNavigator();


function TabIcon({focused, source}: { focused: boolean, source: ImageSourcePropType }) {
    return (

        <View style={{
            width: 40,
            height: 40,
            borderRadius: 20,
            alignItems: "center",
            justifyContent: "center",
            backgroundColor: focused ? 'white' : blue
        }}>
            <Image source={source}
                   style={{
                       tintColor: focused ? blue : 'white',
                       width: 24,
                       height: 24,
                       resizeMode: "stretch",
                   }}/>
        </View>
    )
}

export default function MainTabs() {
    const dispatch = useAppDispatch();

    // Runs after login and on app start with a saved token: the badge and buttons show the server
cart
    useEffect(() => {
        dispatch(fetchCart());
    }, [dispatch]);

    return (
```

```
        <Tab.Navigator
        initialRouteName={"Home"}
        screenOptions={{
            headerShown: false, tabBarLabelVisibilityMode: "unlabeled", tabBarStyle: {
                backgroundColor: blue,
                height: 80,
                borderTopLeftRadius: 16,
                borderTopRightRadius: 16,
                paddingTop: 8
            }
        }}>
        <Tab.Screen name={"Home"} component={HomeTab} options={{
            tabBarIcon:
                ({focused}) => <TabIcon focused={focused}
                                        source={require("../../assets/images/ic_home.png")}/>
        }}/>
        <Tab.Screen name={"Categories"} component={CategoriesTab} options={{
            tabBarIcon:
                ({focused}) => <TabIcon focused={focused}
                                        source={require("../../assets/images/ic_category.png")}/>
        }}/>
        <Tab.Screen name={"WishList"} component={WishListTab} options={{
            tabBarIcon:
                ({focused}) => <TabIcon focused={focused}
                                        source={require("../../assets/images/ic_wishlist.png")}/>
        }}/>
        <Tab.Screen name={"Profile"} component={ProfileTab} options={{
            tabBarIcon:
                ({focused}) => <TabIcon focused={focused}
                                        source={require("../../assets/images/ic_profile.png")}/>
        }}/>


    </Tab.Navigator>)
}
```

2.8 · Logout (ProfileTab.tsx)

Clear the token **and** the cart, so the next user never sees the previous user's cart. Then `reset` the navigation, so the back button can't return to the tabs:

> src/screens/main_tabs/tabs/profile_tab/ProfileTab.tsx

```
import {Pressable, Text, View} from "react-native";
import {useNavigation} from "@react-navigation/native";
import {NativeStackNavigationProp} from "@react-navigation/native-stack";
import type {RootStackParams} from "../../../RootNavigation.tsx";
import {useAppDispatch} from "../../../../store/hooks.ts";
import {clearCart} from "../../../../store/cart/cartSlice.ts";
import {tokenStorage} from "../../../../core/async_storage/asyncStorageHelper.ts";

export default function ProfileTab() {
    const navigation = useNavigation<NativeStackNavigationProp<RootStackParams>>();
    const dispatch = useAppDispatch();

    async function logout() {
        await tokenStorage.clear();      // forget the token
        dispatch(clearCart());           // forget this user's cart
        navigation.reset({index: 0, routes: [{name: "Login"}]});   // back button can't return
    }

    return (<View style={{flex: 1, backgroundColor: '#fff', justifyContent: 'center', alignItems:
'center', gap: 16}}>
        <Text>Profile</Text>
        <Pressable onPress={logout}>
            <Text style={{color: "red", fontWeight: "bold"}}>Logout</Text>
        </Pressable>
    </View>)
}
```

2.9 · When the backend ships the full POST response

Change `addProductToCart` in `cartService.ts` and nothing else:

```
export async function addProductToCart(productId: string): Promise<CartResponse> {
    const response = await apiClient.post<CartResponse>("cart", {productId});
    return response.data;
}
```

To confirm it's safe, call `POST /cart` in Postman and check that each `products[i].product` is an **object** with `_id`, `title` and `imageCover`, not a string.

> **Part 2 checklist**
>
> ☐ `apiServices.ts`: export + token header ☐ `cartService.ts` ☐ replace `cartSlice.ts` ☐ update `AddToCartButton` + `CartScreen` ☐ `fetchCart` in MainTabs ☐ logout in ProfileTab
>
> **Try it:** log in, and the badge shows your server cart (6 for your current account). Add a product: a spinner shows, then the count appears. Open the cart, change counts, pull to refresh. Log out and log in again: the cart reloads from the server.

3 · Reference

Local action → API thunk

Called from screens as	Part 1 (reducer)	Part 2 (thunk → endpoint)
`addToCart(product)`	push line / count + 1, recalculate	`POST /cart` then `GET /cart` (temporary)
`updateCount({productId, count})`	set count, or remove at 0	`PUT /cart/:id`, or `DELETE` at 0
`removeFromCart(productId)`	filter the line out	`DELETE /cart/:id`
`fetchCart()`	-	`GET /cart`
`clearCart()`	reset to empty	reset to empty (local only, on logout)

Flutter → Redux cheat sheet

Flutter	Redux Toolkit
`BlocProvider(create: …)` at the root	`<Provider store={store}>`
`class CartCubit extends Cubit<CartState>`	`createSlice({name: "cart", initialState, reducers})`
`emit(state.copyWith(...))`	"mutate" `state` in a reducer (Immer makes it immutable)
`context.read<CartCubit>().add(p)`	`dispatch(addToCart(p))`
`context.select((CartCubit c) => c.state.count)`	`useAppSelector(state => selectCount(state, id))`
`Future<void> add() async { emit(loading); … emit(loaded) }`	`createAsyncThunk` + `pending` / `fulfilled` / `rejected`
`try { await … } on DioException catch (e)`	`rejectWithValue(getErrorMessage(e))` + `.unwrap().catch(...)`
`BlocListener` for snackbars	`.unwrap().catch(showErrorToast)` in the component

Troubleshooting

Symptom	Cause → fix
"could not find react-redux context value"	The component is outside `<Provider>`. Wrap the whole tree in App.tsx (1.6).
"You are not logged in" toast on every cart action	The token header isn't sent. Apply 2.2, then log in again so a token is saved.
Colors are `undefined` / styles missing in a new component	It imports `blue` from `HomeTab`, a circular import. Import from `src/theme/colors.ts`.
Add fails with a 400 / "invalid id"	A product id isn't a real Route id, for example from the old numeric mock data. Make sure HomeTab uses `getProducts()` from step 1.
Badge shows 2 but the count is 5	Expected: the badge is `numOfCartItems` (different products), like the server. Use `count` sums if you prefer total pieces.
Product card shows **+** for something already in the cart after login	The cart wasn't loaded. Check the `fetchCart()` in MainTabs (2.7).

An existing error in your project: `tsc` reports `Property 'replace' does not exist` at `LoginScreen.tsx:47`. Line 22 uses the screen **props** type where the **navigation** type is needed: `useNavigation<NativeStackScreenProps<RootStackParams>>()` → `useNavigation<NativeStackNavigationProp<RootStackParams>>()`, imported from `@react-navigation/native-stack`.